Day 155 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 23, Day 2 reference

Watch: How does Netflix recommend movies? Matrix Factorization (Luis Serrano Academy) | Matrix Completion and the Netflix Prize (Steve Brunton)

`day155_matrix_factorization.py`, segment by segment, in the order `main()` uses things. The parts (top_svd, low_rank, hard_impute, mf_predict, mf_objective, mf_gradients, als_mf, sgd_mf) are copied unchanged into `day155_exercise.py`; Day 154's parts are imported from `day154_exercise.py`. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: A = a table to factorize · U, s, Vt = its SVD · R, M = ratings and where they are filled · F = the model (mu, bu, bi, P, Q) · P, Q = user and film factors · k = factors · λ = penalty

Segment 0 — Setup

Imports, Day 154's part, settings

CODE

```
import contextlib
import io
import os
import sys
import time
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day154_exercise as d154
LU, LI = 5, 2
ITEM = dict(k=40, shrink=800)
SVD_KS = (1, 2, 3, 5, 10, 20, 50, 100)
MF_KS = (10, 20, 50, 100)
MF_LAMS = (0.05, 0.075, 0.1, 0.125, 0.15)
```

- `sys.path.insert(0, HERE)` — puts this script's folder first on the import search path, so day154_exercise.py is found.
- `with contextlib.redirect_stdout(io.StringIO()):` — everything printed inside the block goes into a string nobody reads: the import's hand checks stay quiet.
- `import day154_exercise as d154` — Day 154's parts, used as d154.load_movielens, d154.fit_biases and so on.
- `ITEM = dict(k=40, shrink=800)` — dict(...) with keyword arguments builds {'k': 40, 'shrink': 800}.

Segment 1 — The parts

top_svd: subspace iteration

MATH

$$B = Q^T A; \quad B\,B^T = W\,diag(s^2)\,W^T; \quad U = Q\,W, \quad V^T = W^T\,B\,/\,s$$

CODE

```
def top_svd(A, k, iters=40, seed=0):
    rng = np.random.RandomState(seed)
    Q = np.linalg.qr(A @ rng.randn(A.shape[1], k + 10))[0]
    for _ in range(iters):
        Q = np.linalg.qr(A @ (A.T @ Q))[0]
    B = Q.T @ A
    w, V = np.linalg.eigh(B @ B.T)
    o = np.argsort(-w)[:k]
    s = np.sqrt(np.maximum(w[o], 0.0))
    return Q @ V[:, o], s, (V[:, o].T @ B) / np.maximum(s, 1e-12)[:, None]
```

- `np.linalg.qr(...)[0]` — QR splits a matrix into orthonormal columns Q times a triangle; [0] keeps Q.
- `A @ (A.T @ Q)` — multiplying by A A^T grows each direction by its squared singular value, so the top ones win.
- `np.linalg.eigh(B @ B.T)` — eigenvalues of a small symmetric (k+10) x (k+10) matrix: the squared singular values.
- `np.argsort(-w)[:k]` — positions of the k largest eigenvalues (eigh returns them smallest first).
- `np.maximum(s, 1e-12)[:, None]` — a column, so each row of V^T B is divided by its own s; never by 0.

low_rank and hard_impute

CODE

```
def low_rank(A, k, seed=0):
    U, s, Vt = top_svd(A, k, seed=seed)
    return (U * s) @ Vt
def hard_impute(A, M, k, iters=30, seed=0):
    Z = np.where(M, A, 0.0)
    for _ in range(iters):
        L = low_rank(Z, k, seed)
        Z = np.where(M, A, L)
    return L
```

- `(U * s) @ Vt` — U * s multiplies column j of U by s[j] (broadcasting), the same as U @ diag(s) without building diag(s).
- `np.where(M, A, 0.0)` — takes A where M is True and 0 elsewhere: the starting fill.
- `np.where(M, A, L)` — known cells keep the real values; empty cells take the latest fit.

mf_predict, mf_objective, mf_gradients

MATH

$$J = \Sigma_{known}\, e_{ui}^{\,2} + \lambda\, \Sigma_u\, n_u(|p_u|^2 + b_u^{\,2}) + \lambda\, \Sigma_i\, n_i(|q_i|^2 + b_i^{\,2})$$

$$\partial J/\partial P = -2\, E\, Q + 2\lambda\, n_u\, P; \quad \partial J/\partial b_u = -2\, \Sigma_i\, e_{ui} + 2\lambda\, n_u\, b_u$$

CODE

```
def mf_predict(F, u, i):
    return F["mu"] + F["bu"][u] + F["bi"][i] + (F["P"][u] * F["Q"][i]).sum(1)
def mf_objective(F, R, M, lam):
    pred = F["mu"] + F["bu"][:, None] + F["bi"][None, :] + F["P"] @ F["Q"].T
    nu, ni = M.sum(1), M.sum(0)
    sq = ((R - pred) ** 2 * M).sum()
    pen = lam * (nu @ ((F["P"] ** 2).sum(1) + F["bu"] ** 2) + ni @ ((F["Q"] ** 2).sum(1) + F["bi"]
        ** 2))
    return sq + pen
def mf_gradients(F, R, M, lam):
    E = (R - F["mu"] - F["bu"][:, None] - F["bi"][None, :] - F["P"] @ F["Q"].T) * M
    nu, ni = M.sum(1), M.sum(0)
    return {"P": -2 * E @ F["Q"] + 2 * lam * nu[:, None] * F["P"],
            "Q": -2 * E.T @ F["P"] + 2 * lam * ni[:, None] * F["Q"],
            "bu": -2 * E.sum(1) + 2 * lam * nu * F["bu"],
            "bi": -2 * E.sum(0) + 2 * lam * ni * F["bi"]}
```

- `F["P"][u] * F["Q"][i]).sum(1)` — one row of P and one row of Q per pair, multiplied and summed along each row: many dot products at once.
- `F["P"] @ F["Q"].T` — every user's factors times every film's: a full users x films table.
- `nu @ ((F["P"] ** 2).sum(1) + ...)` — a dot product with the rating counts: each user's squared sizes counted once per rating.
- `return {"P": ..., "Q": ...}` — a dictionary of gradients with the same keys as the model, so they can be compared entry by entry.

als_mf: alternating least squares

MATH

$$[p_u ; b_u] = (X^T X + \lambda n_u I)^{-1} X^T (r_u - \mu - b_i)$$

CODE

```
def als_mf(R, M, k, lam, sweeps=15, seed=0, track=None):
    rng = np.random.RandomState(seed); nu, ni = M.shape
    F = dict(mu=R[M].mean(), P=0.1 * rng.randn(nu, k), Q=0.1 * rng.randn(ni, k), bu=np.zeros(nu),
        bi=np.zeros(ni))
    hist = []
    for _ in range(sweeps):
        for a in range(nu):
            j = np.flatnonzero(M[a]); X = np.c_[F["Q"][j], np.ones(len(j))]
            y = R[a, j] - F["mu"] - F["bi"][j]
            w = np.linalg.solve(X.T @ X + lam * len(j) * np.eye(k + 1), X.T @ y)
            F["P"][a], F["bu"][a] = w[:k], w[k]
        for b in range(ni):
            j = np.flatnonzero(M[:, b])
            if len(j) == 0:
                F["Q"][b], F["bi"][b] = 0.0, 0.0
                continue
            X = np.c_[F["P"][j], np.ones(len(j))]; y = R[j, b] - F["mu"] - F["bu"][j]
            w = np.linalg.solve(X.T @ X + lam * len(j) * np.eye(k + 1), X.T @ y)
            F["Q"][b], F["bi"][b] = w[:k], w[k]
        hist.append((mf_objective(F, R, M, lam),) + (track(F) if track else ()))
    return F, hist
```

- `np.flatnonzero(M[a])` — the positions of the films user a rated.
- `np.c_[F["Q"][j], np.ones(len(j))]` — glues a column of 1s to the film factors, so the user's bias is solved with the factors.
- `np.linalg.solve(X.T @ X + lam * len(j) * np.eye(k + 1), X.T @ y)` — the ridge regression, solved directly (never by inverting).
- `F["P"][a], F["bu"][a] = w[:k], w[k]` — the first k numbers are the factors, the last is the bias.
- `continue` — skips to the next film; a film nobody rated keeps zeros.
- `(mf_objective(...),) + (track(F) if track else ())` — a one-item tuple joined to whatever the optional track function returns.

sgd_mf: mini-batch stochastic gradient descent

MATH

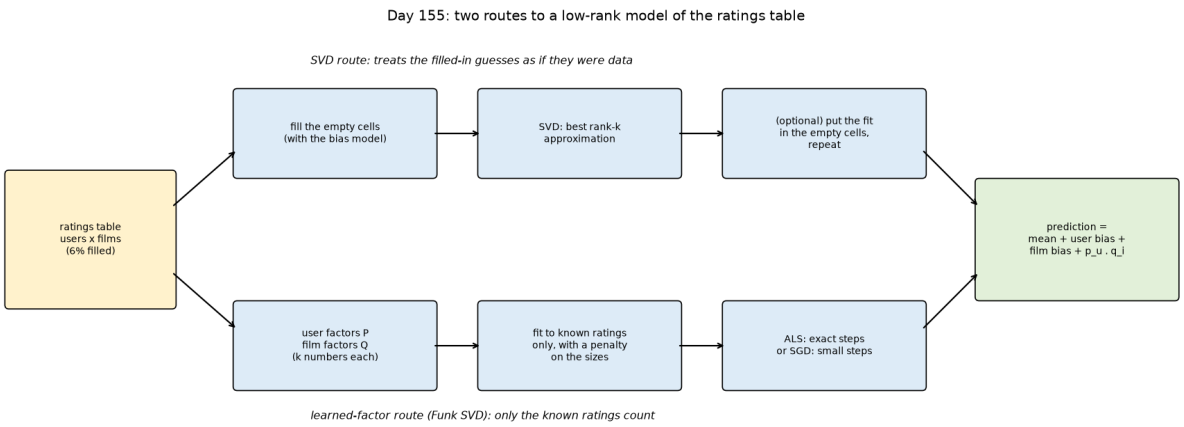
$$e = r - pred; \quad p_u \mathrel{+}= \eta(e\, q_i - \lambda\, p_u); \quad q_i \mathrel{+}= \eta(e\, p_u - \lambda\, q_i)$$

CODE

```
def sgd_mf(u, i, r, nu, ni, k, lam, lr=0.01, epochs=60, batch=128, seed=0, track=None):
    rng = np.random.RandomState(seed)
    F = dict(mu=r.mean(), P=0.1 * rng.randn(nu, k), Q=0.1 * rng.randn(ni, k), bu=np.zeros(nu),
        bi=np.zeros(ni))
    hist = []
    for _ in range(epochs):
        o = rng.permutation(len(r))
        for s in range(0, len(o), batch):
            b = o[s:s + batch]; a, c = u[b], i[b]
            e = r[b] - mf_predict(F, a, c)
            gp = e[:, None] * F["Q"][c] - lam * F["P"][a]
            gq = e[:, None] * F["P"][a] - lam * F["Q"][c]
            np.add.at(F["bu"], a, lr * (e - lam * F["bu"][a]))
            np.add.at(F["bi"], c, lr * (e - lam * F["bi"][c]))
            np.add.at(F["P"], a, lr * gp); np.add.at(F["Q"], c, lr * gq)
        hist.append(track(F) if track else ())
    return F, hist
```

- `rng.permutation(len(r))` — a fresh random order of the ratings every epoch.
- `for s in range(0, len(o), batch):` — start positions 0, 128, 256, ...; the last batch may be shorter.
- `e[:, None] * F["Q"][c]` — each rating's error times its film's factors: one row per rating in the batch.
- `np.add.at(F["P"], a, lr * gp)` — adds every row of lr * gp to row a of P, even when a user appears twice; F['P'][a] += ... would keep only one of the repeats.

SYSTEM DIAGRAM



Two routes to a low-rank model.

Segment 2 — The experiments

Helpers: setup, the bias model, scoring, top-10

CODE

```
def base_model(R, M):
    mu, bu, bi = d154.fit_biases(R, M, LU, LI)
    return mu + bu[:, None] + bi[None, :]
def score(p, r):
    return d154.rmse(np.clip(p, 1, 5), r)
def top10(Q, M, u, i, r, te):
    Q = np.where(M, -np.inf, Q); liked = np.zeros(M.shape, bool); g = r[te] >= 4; liked[u[te][g],
        i[te][g]] = True
    who = np.flatnonzero(liked.sum(1) > 0); top = np.argsort(-Q, axis=1, kind="stable")[:, :10]
    return np.take_along_axis(liked, top, 1)[who].sum(1).mean() / 10,
        np.median(M.sum(0)[top[who]])
```

- `mu + bu[:, None] + bi[None, :]` — a column plus a row broadcasts to a full users x films table.
- `np.clip(p, 1, 5)` — predictions outside 1–5 stars are pulled to the nearest end before scoring.
- `np.median(M.sum(0)[top[who]])` — the rating counts of every recommended film, then their median.

demo_lowrank: the check and the shuffled control

CODE

```
t0 = time.time(); U, s, Vt = top_svd(A, 20); t1 = time.time() - t0
s_np = np.linalg.svd(A, compute_uv=False)
v = A[M]; rng = np.random.RandomState(0); Ash = np.zeros_like(A)
Ash[M] = v[rng.permutation(len(v))]
s_sh = np.linalg.svd(Ash, compute_uv=False)
tot, tsh = (s_np ** 2).sum(), (s_sh ** 2).sum()
```

- `np.linalg.svd(A, compute_uv=False)` — numpy's full SVD, singular values only: the reference to check against.
- `Ash[M] = v[rng.permutation(len(v))]` — the same known values in a shuffled order, put back into the same cells.

demo_als: the gradient check and the grid

MATH

$$\partial J/\partial\theta \approx (J(\theta + h) - J(\theta - h)) / 2h, \quad h = 10^{-6}$$

CODE

```
for q in ("P", "Q", "bu", "bi"):
    for idx in np.ndindex(F[q].shape):
        o = F[q][idx]; F[q][idx] = o + 1e-6; fp = mf_objective(F, Rs, Ms, 0.3); F[q][idx] = o -
            1e-6
        fm = mf_objective(F, Rs, Ms, 0.3); F[q][idx] = o; worst = max(worst, abs((fp - fm) / 2e-6
            - G[q][idx]))
a, b = np.unravel_index(np.argmin(E), E.shape); k, lam = MF_KS[a], MF_LAMS[b]
```

- `np.ndindex(F[q].shape)` — every index tuple of an array of that shape, e.g. (0, 0), (0, 1), ...
- `(fp - fm) / 2e-6` — the central difference: the slope measured by nudging one number up and down.
- `np.unravel_index(np.argmin(E), E.shape)` — the row and column of the smallest validation error.

demo_sgd: early stopping

CODE

```
vtrack = lambda F: (score(mf_predict(F, u[fi], i[fi]), r[fi]), score(mf_predict(F, u[va], i[va]),
    r[va]))
Hs = curves["SGD, lam " + format(lam, "g")][0]; ep = int(np.argmin(Hs[:, 1])) + 1
R, M = d154.to_matrix(u[tr], i[tr], r[tr], nu, ni)
F, _ = sgd_mf(u[tr], i[tr], r[tr], nu, ni, k, lam, epochs=ep)
```

- `lambda F: (..., ...)` — a small unnamed function returning training and validation RMSE; sgd_mf calls it after every epoch.
- `int(np.argmin(Hs[:, 1])) + 1` — the epoch with the lowest validation error (counted from 1), used for the refit.

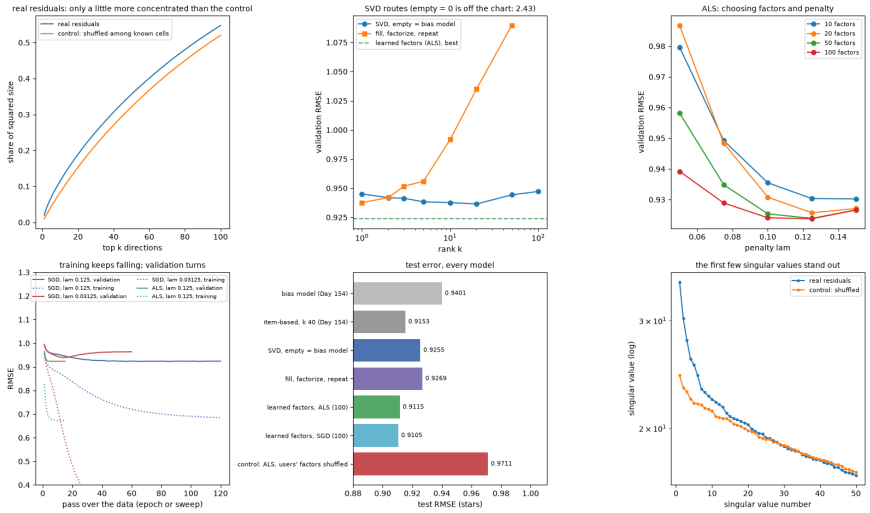
demo_compare: the control and the factor directions

CODE

```
Fsh = dict(Fals); Fsh["P"] = Fals["P"][np.random.RandomState(0).permutation(nu)]
_, _, Vt = np.linalg.svd(Q[ok] - Q[ok].mean(0), full_matrices=False)
sw = titles.index("Star Wars (1977)"); qn = Q / np.maximum(np.linalg.norm(Q, axis=1,
    keepdims=True), 1e-12)
sim = qn[ok] @ qn[sw]; o = [j for j in np.argsort(-sim) if ok[j] != sw][:5]
```

- `dict(Fals)` — a shallow copy: replacing Fsh['P'] leaves the trained model untouched.
- `np.linalg.svd(Q[ok] - Q[ok].mean(0), full_matrices=False)` — the main directions of the centred film factors (a PCA).
- `np.linalg.norm(Q, axis=1, keepdims=True)` — each row's length, kept as a column so the division broadcasts.
- `qn[ok] @ qn[sw]` — dot products of unit vectors: the cosine of the angle with Star Wars.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `contextlib.redirect_stdout(io.StringIO())` — silence printing inside a with block.
- `np.linalg.qr(X)[0]` — orthonormal columns spanning X.
- `np.linalg.eigh(S)` — eigenvalues and vectors of a symmetric matrix, smallest first.

- `(U * s) @ Vt` — U diag(s) Vt without building the diagonal matrix.
- `np.where(M, A, L)` — choose between two arrays cell by cell.
- `np.c_[X, np.ones(n)]` — append a column.
- `np.linalg.solve(A, b)` — solve A x = b.
- `np.add.at(a, idx, v)` — add with repeated indices counted.
- `np.ndindex(shape)` — every index tuple of a shape.
- `dict(F)` — a shallow copy of a dictionary.

End of Day 155 syntax reference. For the experiments and what they showed, see the Day 155 complete guide.